

CONVOLUTIONAL NEURAL NETWORKS

Architecture, Mathematics & Real-World Applications

Rahul Biswas, Roll no. 16

Table of Contents

Table of Contents

1. Introduction to Convolutional Neural Networks.....	4
1.1 Brief History and Motivation.....	4
1.2 Why CNNs Outperform Traditional Networks on Images	4
1.3 Core Building Blocks at a Glance	5
3. Stride and Padding	6
3.1 Stride.....	6
3.2 Output Size Formula	7
3.3 Padding.....	7
4. End-to-End CNN Architecture	8
4.1 Layer-by-Layer Breakdown	8
4.2 Receptive Field	9
5. Activation Functions.....	10
5.1 ReLU — Rectified Linear Unit.....	10
5.2 Variants of ReLU.....	11
5.3 Softmax — Output Activation.....	11
6. Pooling Layers	11
6.1 Comparison of Pooling Strategies	12
7. Training Convolutional Neural Networks	12
7.1 Loss Functions	12
7.2 Optimization Algorithms.....	13
7.3 Regularization Techniques	13
8. Landmark CNN Architectures	14
8.1 LeNet-5 (1998)	14
8.2 AlexNet (2012)	14
8.3 VGGNet (2014)	14
8.4 GoogLeNet / Inception (2014)	15
8.5 ResNet (2015).....	15
8.6 EfficientNet (2019).....	15
9. Transfer Learning	16
9.1 Fine-tuning Strategies	16
9.2 Popular Pre-trained Models for Transfer Learning	16
10. Real-World Applications of CNNs	18
10.1 Autonomous Vehicles & Intelligent Mobility	18
10.2 Healthcare & Medical Imaging.....	18
10.3 E-Commerce & Visual Search	19

10.4 Security & Biometrics	19
10.5 Precision Agriculture	20
11. Challenges, Limitations & Future Directions.....	21
11.1 Current Challenges	21
11.2 Emerging Solutions	21
11.3 The Rise of Vision Transformers	21
12. CNN Architecture Comparison Table	23
Glossary of Key Terms	24
References & Further Reading	26

1. Introduction to Convolutional Neural Networks

A Convolutional Neural Network (CNN) is a specialized class of deep neural networks designed primarily to process and analyze spatial data, such as digital images. In traditional neural networks, every pixel of an input image connects to every single neuron in the subsequent layer — a fully connected approach that ignores spatial structure and causes a massive explosion in learnable parameters.

CNNs overcome this by looking at images in small localized patches, seamlessly capturing hierarchical visual features like edges, textures, shapes, and complex object parts. The word "convolutional" comes from the mathematical operation of convolution, which is at the heart of how CNNs extract features from raw pixel data.

1.1 Brief History and Motivation

The concept of CNNs dates back to the late 1980s, when Yann LeCun developed LeNet-5 for handwritten digit recognition. The architecture remained largely academic until 2012, when AlexNet — designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton — dramatically outperformed classical computer-vision methods on the ImageNet benchmark, reducing the top-5 error rate by over 10 percentage points. That breakthrough ignited the modern deep-learning revolution.

1.2 Why CNNs Outperform Traditional Networks on Images

- Parameter sharing: the same filter weights are reused across spatial positions, drastically reducing the total parameter count.
- Local connectivity: neurons respond to features in a small receptive field, respecting spatial locality.
- Translation invariance: features detected in one region can be recognized anywhere in the image.
- Hierarchical feature learning: shallow layers detect edges; deeper layers detect textures, parts, and objects.

1.3 Core Building Blocks at a Glance

Component	Function	Key Hyper-parameters
Convolutional Layer	Extracts local feature maps via learned filters	Kernel size, # filters, stride, padding
Activation (ReLU)	Introduces non-linearity	Threshold (0 for standard ReLU)
Pooling Layer	Down samples feature maps	Pool size, stride, type (max/avg)
Batch Normalization	Stabilizes and accelerates training	Momentum, epsilon
Dropout	Regularization by random neuron deactivation	Drop probability
Fully Connected	High-level reasoning and classification	neurons, activation

2. The Convolution Operation

In deep learning, convolution is a mathematical operation that merges two entities: the input data (an image or feature map) and a small learnable matrix called a kernel or filter. Visually, think of convolution as sliding a small window across a large picture and at each step evaluating how strongly a particular pattern (edge, curve, colour gradient) is present underneath that window.

The result is a Feature Map (also called an activation map) that encodes the spatial presence and strength of the detected feature across the image.

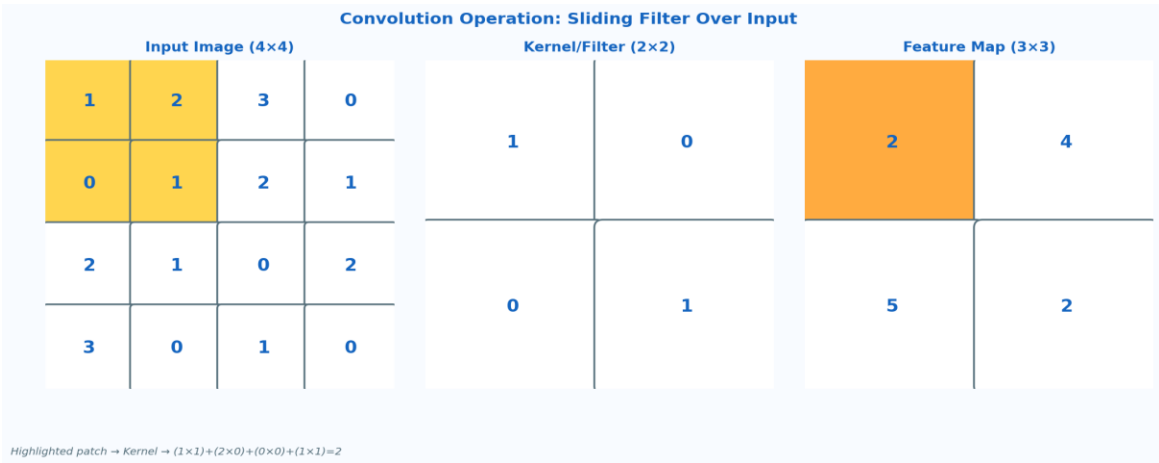


Figure 1: Convolution Operation — a 2x2 kernel slides over a 4x4 input producing a Feature Map. The highlighted top-left patch yields output value 2.

3. Stride and Padding

Two critical hyper-parameters determine the spatial dimensions of feature maps and how thoroughly the filter samples the input: Stride and Padding.

3.1 Stride

Stride specifies how many pixels the filter shifts horizontally or vertically between each application. It directly controls the degree of overlap between filter positions.

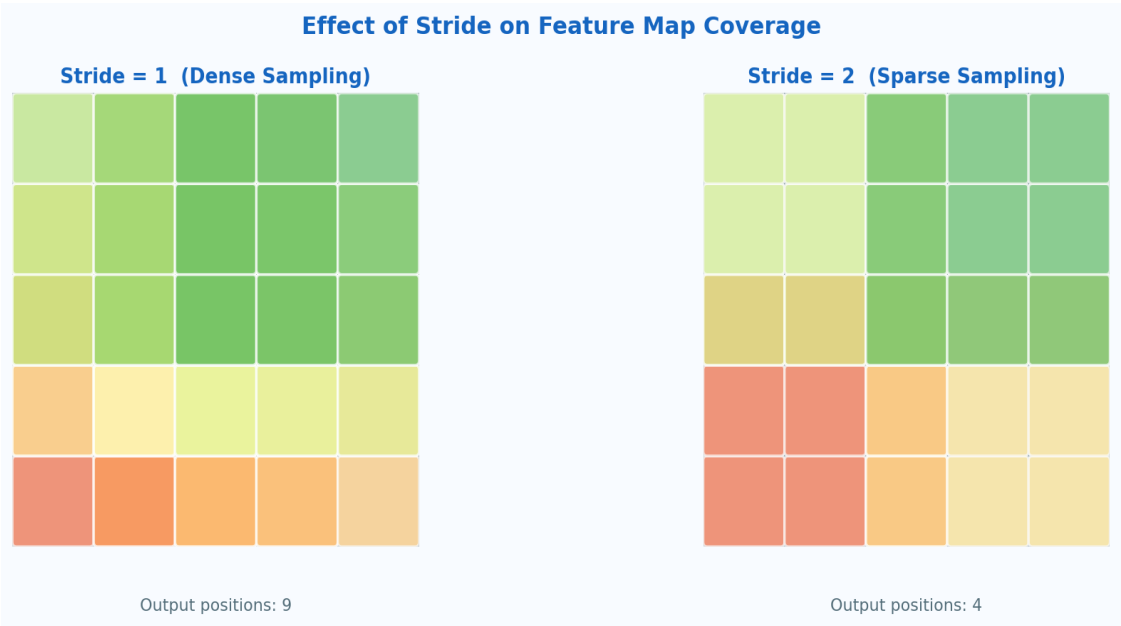


Figure 2: Stride Comparison — Stride=1 (left) yields dense, overlapping coverage; Stride=2 (right) downsamples by skipping alternate positions.

Stride Value	Filter Movement	Output Size	Use Case
1	One pixel at a time	Largest output (W-F+1)	Fine-grained feature extraction
2	Two pixels at a time	Roughly halved	Spatial downsampling (replaces pooling)
3+	Larger hops	Highly reduced	Very aggressive compression (rarely used)

3.2 Output Size Formula

$$\text{Output Size} = \lfloor (W - F + 2P) / S \rfloor + 1$$

Where: W = input dimension, F = filter size, P = padding amount, S = stride. For example, a 32×32 input with a 3×3 filter, padding=1, stride=1 yields: $\lfloor (32-3+2)/1 \rfloor + 1 = 32$ (same-size output).

3.3 Padding

Padding adds a border of zeros (zero-padding, also called "same" padding) or reflects actual pixel values around the input before applying convolution. Without padding, the filter cannot overlap the border pixels as extensively, causing the output feature map to shrink and edge features to be under-represented.

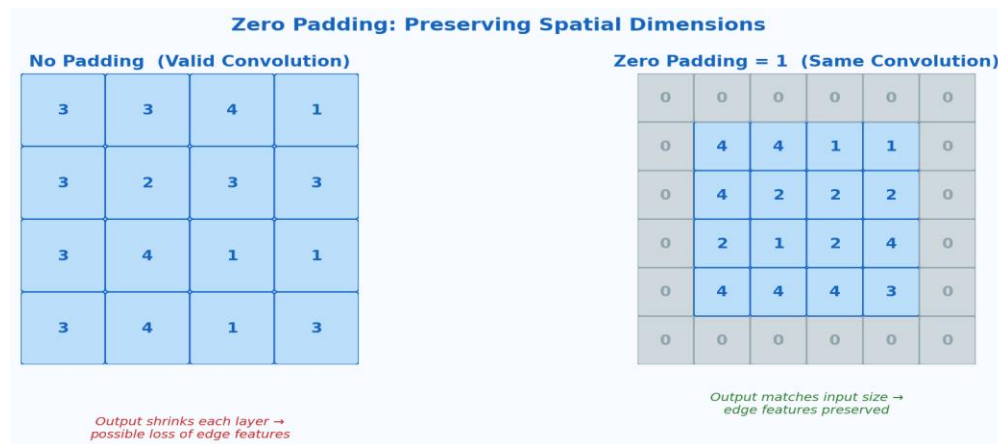


Figure 3: Zero Padding — No padding (left) shrinks the feature map each layer; padding=1 (right) preserves spatial dimensions and treats edge pixels equally.

Padding Type	Description	Effect on Output Size
Valid (no padding)	Filter applied only where it fully fits	Output < Input
Same (zero padding)	Zeros added to preserve spatial dimensions	Output = Input (for stride=1)
Full padding	Maximum zeros to let filter touch every pixel	Output > Input

4. End-to-End CNN Architecture

A complete CNN stacks multiple layer types in a deliberate sequence. Early layers detect primitive features (edges, colour blobs); intermediate layers combine these into textures and object parts; and the final layers perform classification or regression over the entire image.

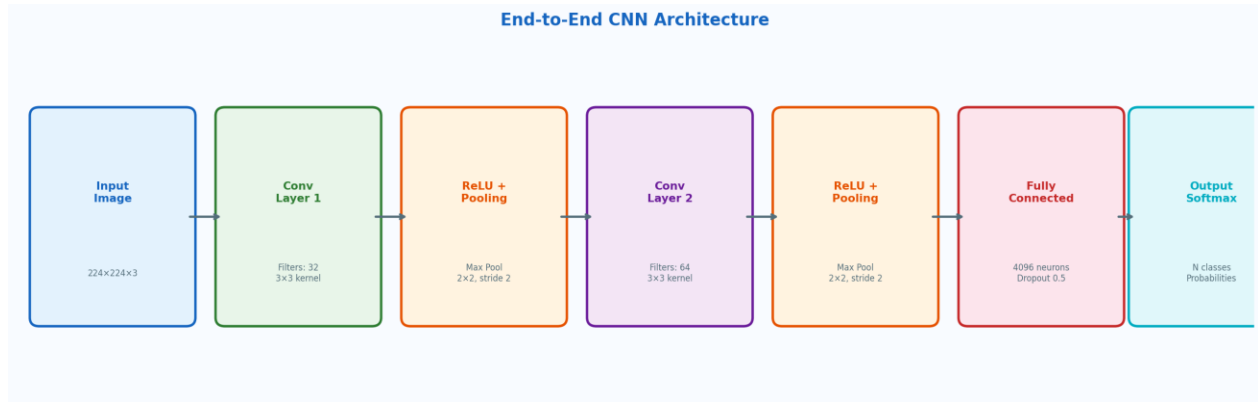


Figure 4: Full CNN pipeline — Input → Conv+ReLU blocks → Pooling → Fully Connected → Softmax output.

4.1 Layer-by-Layer Breakdown

Layer	Primary Function	Conceptual Purpose	Output Shape Example
Input Layer	Holds raw pixel intensities	Accepts the original image tensor	224×224×3
Conv Layer 1	Detects low-level features	Edge, colour gradient detection	222×222×32
Activation (ReLU)	Introduces non-linearity	Replaces negatives with 0	222×222×32
Pooling Layer 1	Spatial down sampling	Halves spatial dimensions	111×111×32
Conv Layer 2	Detects mid-level features	Texture, shape patterns	109×109×64
Activation (ReLU)	Introduces non-linearity	Keeps only positive activations	109×109×64
Pooling Layer 2	Further down sampling	Compresses spatial info	54×54×64

Flatten	Converts 3D to 1D	Prepares for dense layers	186624
Fully Connected	High-level reasoning	Maps features to class scores	4096
Output (Softmax)	Classification probabilities	Sums to 1.0 across all classes	N classes

4.2 Receptive Field

The receptive field of a neuron is the region of the input image that influences its activation. In shallow layers, the receptive field is small (a few pixels). As the network deepens, the receptive field grows because pooling and multiple convolution operations progressively aggregate information from larger spatial regions. By the final convolutional layer, each neuron may have a receptive field covering most or all of the input image.

5. Activation Functions

Activation functions introduce non-linearity into the network. Without them, stacking linear operations (convolutions, matrix multiplications) would collapse to a single affine transformation, no matter how many layers were used. Non-linear activations allow CNNs to approximate arbitrarily complex functions.

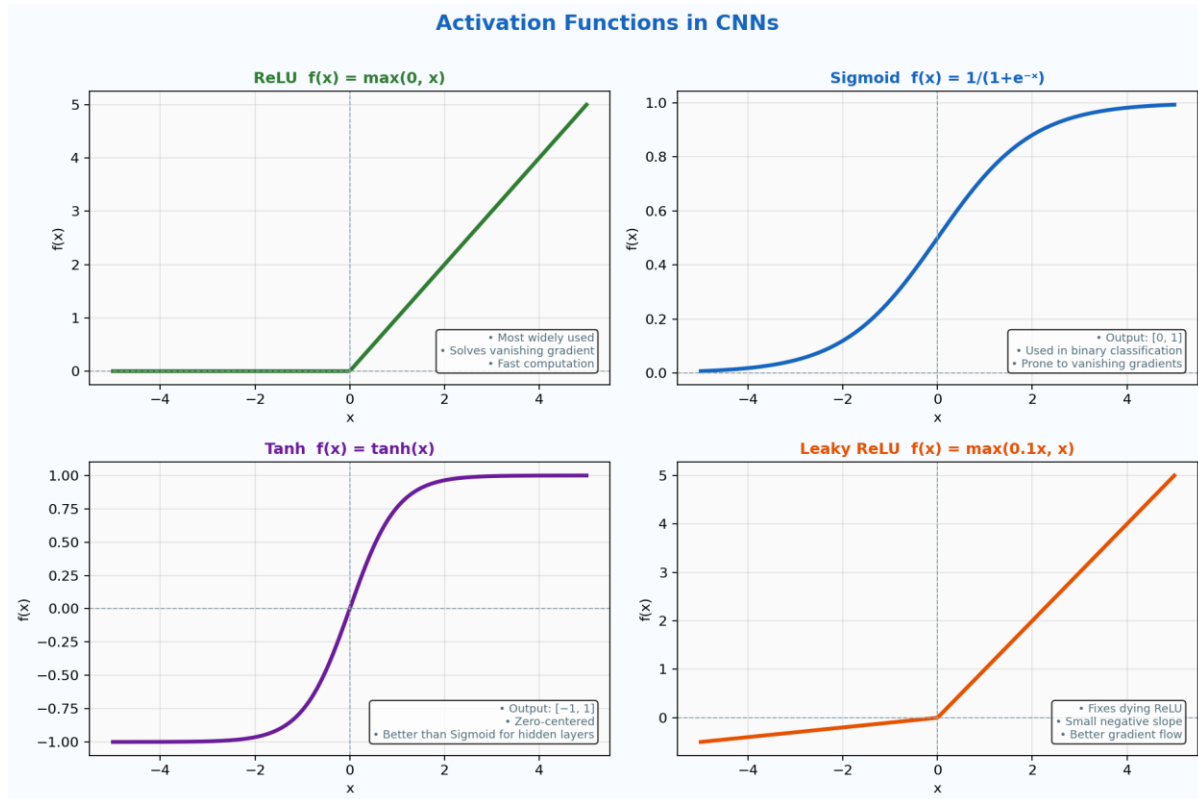


Figure 5: Four common activation functions plotted on the range $[-5, 5]$. ReLU is the de-facto standard for hidden layers; Softmax is used at the output for multi-class classification.

5.1 ReLU — Rectified Linear Unit

ReLU is the most widely used activation in modern CNNs. It is computationally cheap, does not suffer from the vanishing gradient problem for positive activations, and empirically converges faster than sigmoid or tanh.

$$f(x) = \max(0, x)$$

Drawback: "Dying ReLU" — neurons can permanently output 0 if they receive consistently negative inputs during training.

5.2 Variants of ReLU

Variant	Formula	Advantage Over Standard ReLU
Leaky ReLU	$f(x) = \max(\alpha x, x), \alpha=0.01$	Small negative slope prevents dying neurons
Parametric ReLU	$f(x) = \max(\alpha x, x), \alpha$ learned	α is a trainable parameter per channel
ELU	x if $x>0$ else $\alpha(e^x-1)$	Negative values push mean activations closer to 0
GELU	$x \cdot \Phi(x)$	Used in Transformers; smoother than ReLU

5.3 Softmax — Output Activation

For multi-class classification, the final layer applies Softmax, which normalizes raw scores (logits) into a probability distribution summing to 1.0:

$$\text{Softmax}(z_i) = e^{z_i} / \sum_j e^{z_j}$$

6. Pooling Layers

Pooling (also called sub-sampling) reduces the spatial dimensions of feature maps, decreasing both computational cost and the risk of over-fitting by introducing a degree of spatial invariance — small translations of a feature in the input will not drastically change the pooled output.

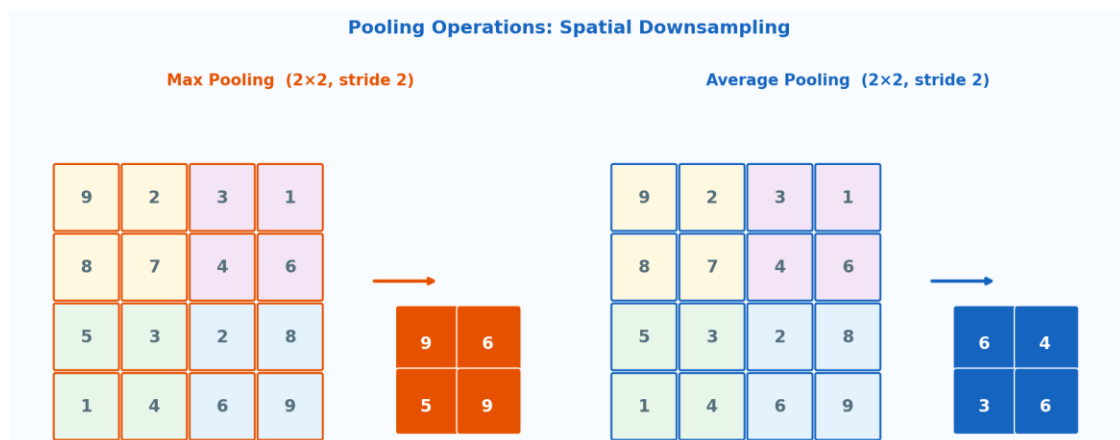


Figure 6: Pooling Operations — Max Pooling selects the largest value per region (preserves dominant features); Average Pooling computes the mean (smoother representation).

6.1 Comparison of Pooling Strategies

Type	Operation	Typical Use Case	Pros / Cons
Max Pooling	Takes maximum value in the window	Most CNN architectures	Preserves sharp features / Discards non-max info
Average Pooling	Computes mean over the window	Final summary layer (GAP)	Smooth output / May blur sharp edges
Global Average Pooling (GAP)	Single average over entire map	Replace FC layers (ResNet, MobileNet)	Fewer parameters, less overfitting / Information loss
Fractional Max Pooling	Non-integer stride using random grids	Regularization experiments	Stochastic / Complex implementation

7. Training Convolutional Neural Networks

Training a CNN involves iteratively adjusting its millions of learnable parameters (filter weights and biases) so that its predictions on training data become increasingly accurate. This is done via a process called Backpropagation combined with gradient-based optimization.

7.1 Loss Functions

The loss function quantifies how far the model's predictions are from the ground truth labels. Minimizing the loss is the training objective.

Loss Function	Formula (simplified)	When to Use
Cross-Entropy Loss	$L = -\sum y_i \log(\hat{y}_i)$	Multi-class classification (Softmax output)
Binary Cross-Entropy	$L = -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$	Binary classification (Sigmoid output)
Mean Squared Error	$L = (1/N) \sum (y - \hat{y})^2$	Regression tasks
Focal Loss	$L = -\alpha(1-p_t)^{\gamma} \log(p_t)$	Class-imbalanced detection problems

7.2 Optimization Algorithms

Optimizer	Core Idea	Learning Rate Behaviour
SGD	Update weights in direction of negative gradient	Fixed (manual scheduling needed)
SGD + Momentum	Accumulates velocity from previous gradients	Faster convergence in ravines
Adam	Adaptive learning rates per parameter	Automatically decays; popular default
AdamW	Adam + decoupled weight decay	Better generalization than Adam
RMSprop	Divides gradient by running mean of magnitudes	Good for RNNs and noisy gradients

7.3 Regularization Techniques

- Dropout: randomly sets a fraction p of neuron activations to 0 during training, forcing the network to learn redundant representations.
- Weight Decay (L2 Regularization): adds a penalty proportional to the sum of squared weights to the loss, discouraging excessively large weights.
- Data Augmentation: artificially expands the training set via random flips, rotations, crops, colour jitter, and mixup, reducing overfitting.
- Batch Normalization: normalizes layer inputs to zero mean and unit variance, allowing higher learning rates and acting as a mild regularizer.
- Early Stopping: monitors validation loss and halts training when it stops improving, preventing the model from memorizing training noise.

8. Landmark CNN Architectures

Over the past decade, a series of landmark architectures have pushed the boundaries of image recognition accuracy, training efficiency, and deployability. Each introduced a key architectural innovation that inspired an entire sub-field of research.

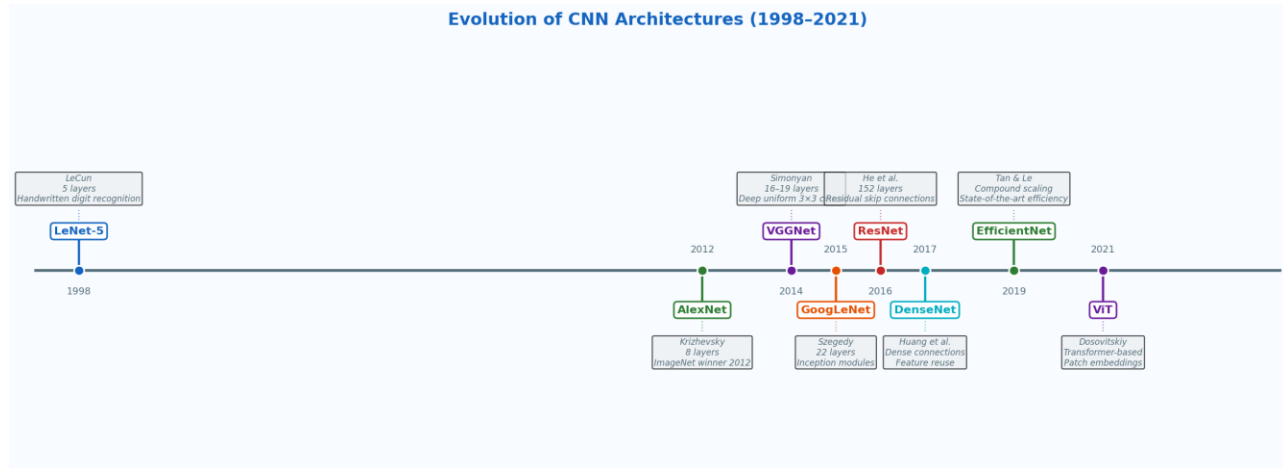


Figure 7: Evolution of CNN architectures from LeNet-5 (1998) to Vision Transformer (2021).

8.1 LeNet-5 (1998)

Authors: Yann LeCun, Leon Bottou, Yoshua Bengio, Patrick Haffner

LeNet-5 was the first successful deep CNN, trained for handwritten digit recognition on the MNIST dataset. It introduced the now-standard pattern of alternating convolutional and pooling layers followed by fully connected layers. Despite being only 5 layers deep with 60,000 parameters, it achieved near-human accuracy on digit classification.

8.2 AlexNet (2012)

Authors: Alex Krizhevsky, Ilya Sutskever, Geoffrey Hinton

AlexNet achieved a top-5 error of 15.3% on ImageNet 2012 — nearly 11 percentage points better than the runner-up. Key innovations: GPU training across two cards, ReLU activations (replacing tanh), Local Response Normalization, dropout regularization ($p=0.5$), and heavy data augmentation.

8.3 VGGNet (2014)

Authors: Karen Simonyan, Andrew Zisserman (Oxford)

VGG demonstrated that network depth with very small (3×3) convolution filters is a critical component for achieving strong performance. VGG-16 (16 weight layers) and VGG-19 are still widely used as feature extractors due to their simplicity and strong pre-trained weights.

8.4 GoogLeNet / Inception (2014)

Authors: Christian Szegedy et al. (Google)

GoogLeNet introduced the Inception module: a parallel combination of 1×1, 3×3, and 5×5 convolutions plus pooling, whose outputs are concatenated along the channel dimension. This multi-scale feature extraction reduced computation while increasing representational power. At 22 layers and only 5 million parameters, it was far more efficient than VGGNet.

8.5 ResNet (2015)

Authors: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun (Microsoft Research)

ResNet won ILSVRC 2015 with a top-5 error of 3.57% — surpassing human performance. Its key innovation was the residual (skip) connection: instead of learning $H(x)$, the layer learns the residual $F(x) = H(x) - x$. This allows gradients to flow directly through the network during backpropagation, enabling training of extremely deep networks (152 layers and beyond).

$$H(x) = F(x) + x \quad (\text{Residual Block Identity Mapping})$$

8.6 EfficientNet (2019)

Authors: Mingxing Tan, Quoc V. Le (Google Brain)

EfficientNet introduced compound scaling: simultaneously scaling network width (channels), depth (layers), and resolution (image size) using a fixed ratio determined by neural architecture search. EfficientNet-B7 achieves 84.4% top-1 accuracy on ImageNet while being 8.4x smaller and 6.1x faster than the best existing CNN.

9. Transfer Learning

Transfer learning is a technique in which a model trained on a large dataset (typically ImageNet with 1.2 million images across 1,000 classes) is adapted for a different but related task, often with a much smaller dataset. This leverages the rich feature representations already encoded in the pre-trained model's early and middle layers.

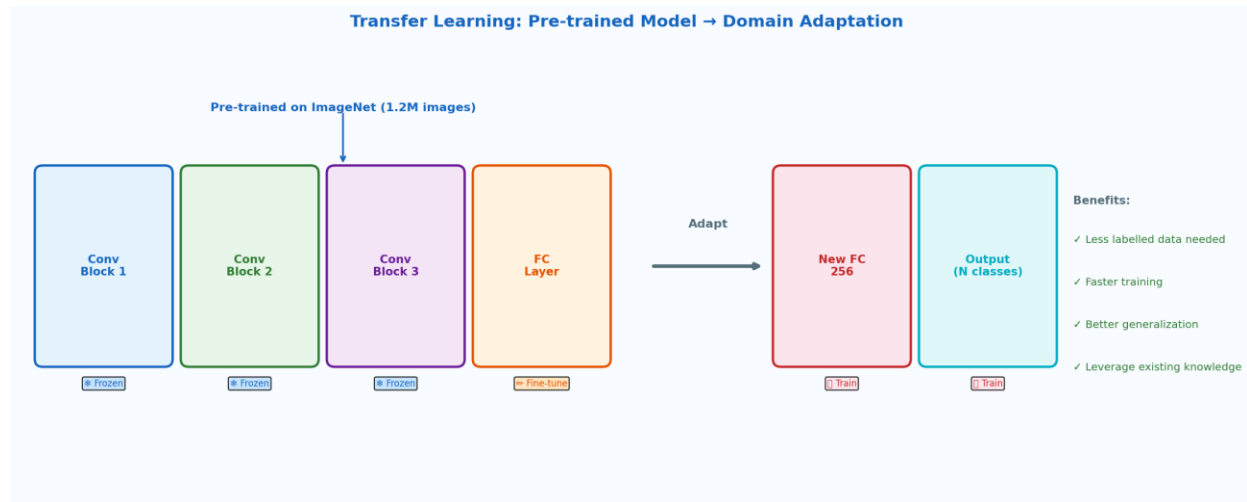


Figure 8: Transfer Learning workflow — early convolutional blocks are frozen; only the classification head is re-trained on the target dataset.

9.1 Fine-tuning Strategies

Strategy	Layers Frozen	New Layers	Best For
Feature Extraction	All conv layers	New FC head only	Very small dataset (< 1,000 samples)
Partial Fine-tuning	Early conv layers	Later conv + new head	Medium dataset; similar domain
Full Fine-tuning	None	New head + all weights updated	Large dataset; different domain

9.2 Popular Pre-trained Models for Transfer Learning

- ResNet-50 / ResNet-101: robust general-purpose feature extractor, widely supported in all frameworks.
- VGG-16: simple architecture; excellent baseline for fine-tuning experiments.

- InceptionV3: multi-scale receptive fields; ideal when objects vary greatly in size.
- EfficientNet-B0 to B7: best accuracy/efficiency trade-off for mobile and cloud deployments.
- MobileNetV3: optimized for on-device inference with depthwise separable convolutions.
- CLIP (OpenAI): vision-language model enabling zero-shot classification via text prompts.

10. Real-World Applications of CNNs

CNNs are deployed across virtually every domain that involves images, video, or spatially structured data. The following sections detail the major application areas with concrete examples.



Figure 9: CNN application domains — from autonomous vehicles to mobile applications.

10.1 Autonomous Vehicles & Intelligent Mobility

Self-driving vehicles process live high-definition video frames from multiple onboard cameras in real time. CNN architectures perform:

- Lane detection and road segmentation using semantic segmentation networks (e.g., SegNet, DeepLab).
- Pedestrian trajectory estimation and bounding-box prediction via object detection models (YOLO, Faster R-CNN).
- Traffic sign and signal recognition with classification heads fine-tuned on annotated sign datasets.
- Depth estimation from stereo camera pairs using disparity-prediction CNNs.

Companies such as Tesla, Waymo, and Mobileye have deployed CNN-based perception stacks in production vehicles logging billions of miles.

10.2 Healthcare & Medical Imaging

CNNs match or exceed expert radiologists on several diagnostic tasks:

- Chest X-ray pathology detection: CheXNet (121-layer DenseNet) detects 14 conditions including pneumonia, cardiomegaly, and pleural effusion.
- Histopathology: CNNs classify cancer grade from whole-slide images (e.g., Camelyon17 metastasis detection challenge).
- Diabetic retinopathy screening: Google's DeepMind system achieves AUC >0.99 on retinal fundus photographs.
- Brain MRI segmentation: 3D U-Net variants segment tumour sub-regions (enhancing core, oedema, necrosis) in BraTS benchmarks.

10.3 E-Commerce & Visual Search

Visual search enables users to photograph any object and instantly retrieve matching or similar products:

- Pinterest Lens uses a custom ResNet-derived architecture to return visually similar images from billions of indexed pins.
- Amazon StyleSnap matches clothing items by colour, cut, and texture using multi-scale CNN embeddings.
- Shelf-detection CNNs in retail stores track product placement, detect empty shelves, and enable cashier-less checkout (Amazon Go).

10.4 Security & Biometrics

Application	CNN Task	Key Metric	Example System
Face Unlock	Face verification (1:1 match)	False Acceptance Rate < 1/1,000,000	Apple Face ID
Surveillance	Face identification (1:N search)	Rank-1 recognition rate	Clearview AI, NIST FRVT
Iris Recognition	Texture classification	Equal Error Rate < 0.1%	IriCore, EyeLock
Fingerprint Matching	Minutiae detection & matching	FMR/FNMR trade-off	NEC, Aware Inc.

10.5 Precision Agriculture

Drones equipped with multispectral and RGB sensors capture aerial imagery of fields. CNN models analyse these images to:

- Detect early-stage crop diseases by identifying characteristic colour and texture patterns on leaves.
- Estimate plant biomass and yield through canopy segmentation.
- Map weed infestations, enabling precision herbicide application and reducing chemical use by up to 90%.
- Monitor irrigation by identifying water-stress indicators in near-infrared bands.

11. Challenges, Limitations & Future Directions

11.1 Current Challenges

- Data hunger: CNNs require large labelled datasets. Curating such datasets is expensive and time-consuming.
- Interpretability: despite many visualization tools (Grad-CAM, LIME, SHAP), CNN decision processes remain difficult to explain to non-experts.
- Adversarial vulnerability: imperceptible perturbations to input images can cause drastic misclassification.
- Computational cost: large models like ResNet-152 or EfficientNet-B7 require significant GPU memory and inference time, challenging edge deployment.
- Dataset bias: models trained on biased datasets propagate and amplify societal biases, especially in facial analysis systems.

11.2 Emerging Solutions

Challenge	Emerging Solution	Research Status
Data hunger	Self-supervised learning (SimCLR, DINO, MAE)	Production-ready
Interpretability	Grad-CAM, Integrated Gradients, Concept-based explanations	Active research
Adversarial attacks	Adversarial training, certified defenses, input denoising	Partially solved
Computational cost	Knowledge distillation, Neural Architecture Search (NAS), Pruning	Production-ready
Dataset bias	Fairness-aware training, diverse data collection, audit frameworks	Active research

11.3 The Rise of Vision Transformers

Since 2020, Vision Transformers (ViT) have challenged CNN dominance by applying the Transformer self-attention mechanism directly to image patches. ViT and its successors (DeiT, Swin Transformer, ConvNeXt) achieve state-of-the-art results on ImageNet while being more scalable with data. Hybrid

architectures combining CNN feature extractors with Transformer encoders (CvT, LeViT) aim to combine the inductive biases of CNNs with the global context modelling of attention.

12. CNN Architecture Comparison Table

The following table compares key technical metrics across landmark CNN architectures to help practitioners select the right model for their use case.

Architecture	Year	Depth	Parameters	Top-1 Acc. (ImageNet)	Key Innovation
LeNet-5	1998	5	60 K	N/A (MNIST)	First conv+pool pattern
AlexNet	2012	8	61 M	63.3%	GPU training, ReLU, Dropout
VGG-16	2014	16	138 M	74.4%	Uniform 3×3 convolutions
GoogLeNet	2014	22	5 M	74.8%	Inception multi-scale module
ResNet-50	2015	50	25 M	76.1%	Residual skip connections
DenseNet-201	2017	201	20 M	77.4%	Dense connections, feature reuse
MobileNetV2	2018	53	3.4 M	72.0%	Depthwise separable convolutions
EfficientNet-B0	2019	18	5.3 M	77.1%	Compound scaling NAS
EfficientNet-B7	2019	66	66 M	84.4%	Largest EfficientNet variant
ConvNeXt-Base	2022	36	89 M	83.8%	Modernized ResNet-style CNN

Glossary of Key Terms

Term	Definition
Activation Map	Output of a convolutional layer representing the spatial strength of a detected feature.
Backpropagation	Algorithm to compute gradients of the loss with respect to all parameters by applying the chain rule.
Batch Normalization	Technique that normalizes layer inputs across the mini-batch to stabilize and accelerate training.
Convolution	Mathematical operation that slides a learnable filter over input data to extract spatial features.
Dropout	Regularization technique that randomly deactivates neurons during training to reduce overfitting.
Feature Map	The output of a convolutional layer; encodes the presence and location of a specific feature.
Filter / Kernel	Small learnable weight matrix (e.g., 3×3) slid across the input to extract a particular feature.
Fine-tuning	Adapting a pre-trained model to a new task by continuing training on a target dataset.
GAP	Global Average Pooling — averages a feature map to a single value, replacing large FC layers.
Gradient Vanishing	Problem where gradients shrink exponentially during backpropagation, stalling training in early layers.
ImageNet	A benchmark dataset of 1.2 million labelled images across 1,000 categories; standard CNN training set.
Max Pooling	Pooling operation that selects the maximum value in each spatial window of the feature map.
Padding	Adding zeros around the input border to control output feature map dimensions.
Receptive Field	The input image region that influences a particular neuron's activation.

ReLU	Rectified Linear Unit: $f(x) = \max(0, x)$. The most common activation function in CNNs.
Residual Connection	Skip connection that adds the input directly to the output of a block, enabling very deep networks.
Softmax	Output activation that converts raw scores (logits) to a probability distribution summing to 1.
Stride	Number of pixels the filter moves at each step; controls output spatial resolution.
Transfer Learning	Reusing a model trained on one task as the starting point for a related task.

References & Further Reading

The following key works provide foundational and advanced reading on CNN theory and applications:

1. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278-2324.
2. Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *NIPS 2012*.
3. Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556*.
4. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *CVPR 2016*.
5. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Available: deeplearningbook.org
6. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *ICML 2019*.
7. Dosovitskiy, A. et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *ICLR 2021*.
8. Liu, Z. et al. (2022). A ConvNet for the 2020s. *CVPR 2022*. (ConvNeXt)